

THÈSE DE DOCTORAT

Titre français

Davide FISSORE

Centre Inria d'Université Côte d'Azur

**Présentée en vue de l'obtention
du grade de docteur en** Computer Science
d'Université Côte d'Azur

Dirigée par : Jean-Paul HÉVIN, Chocolatier,
Maison Hévin

Co-dirigée par : Antoine SANTOS, Chef
pâtissier, École Criollo

Co-encadrée par : Laurent DUCHÊNE, Chef
pâtissier, Maison Duchêne

Soutenue le : TODO

Devant le jury, composé de :

Paul BOCUSE, Grand chef cuisinier,
L'Auberge du Pont de Collonges

Patrick LENÔTRE, Chef de cuisine, Pavillon
des Princes, Paris

Christophe MICHALAK, Chef pâtissier,
l'Hôtel Plaza-Athénée, Paris

Cédric GROLET, Chef pâtissier, Le Meurice

Michel GUÉRARD, Cuisinier, Les Prés
d'Eugénie

TRITRE FRANCAIS

English Title

Davide FISSORE



Jury :

Président du jury

Paul BOCUSE, Grand chef cuisinier, L'Auberge du Pont de Collonges

Rapporteurs

Patrick LENÔTRE, Chef de cuisine, Pavillon des Princes, Paris

Christophe MICHALAK, Chef pâtissier, l'Hôtel Plaza-Athénée, Paris

Examineurs

Cédric GROLET, Chef pâtissier, Le Meurice

Directeur de thèse

Jean-Paul HÉVIN, Chocolatier, Maison Hévin

Co-directeur de thèse

Antoine SANTOS, Chef pâtissier, École Criollo

Co-encadrant de thèse

Laurent DUCHÊNE, Chef pâtissier, Maison Duchêne

Membres invités

Michel GUÉRARD, Cuisinier, Les Prés d'Eugénie

Davide FISSORE

Tritre francais

xii+22 p.

*À toi lecteur <3 :**

Titre français

Mots-clés : Pâtisserie, Macaron, Ladurée.

English Title

Abstract

Macarons are so good.

Keywords: Pastry, Macaron, Ladurée.

Acknowledgement

Merci !

Contents

1	Introduction	1
1.1	Prologue	1
1.2	Automation is everywhere	1
1.2.1	Probabilistic reasoning: artificial intelligence	1
1.2.2	Automated reasoning: elaboration	1
1.3	Contributions	1
1.3.1	Type-class inference	1
1.3.2	Static determinacy checking	1
2	State of the Art	3
2.1	The Rocq proof assistant	3
2.1.1	The basis of the language	3
2.1.2	Elaboration	3
2.1.3	Ad-hoc polymorphism: Type Classes	3
2.1.4	Concluding remarks	3
2.2	The Elpi language	3
2.2.1	Controlling backtracking: the ! operator	3
2.2.2	Hints about unification	3
2.2.3	Rocq-Elpi: Rocq and Elpi interleaving	3
2.2.4	CHR: goal suspension	3
3	Type classes resolution in Elpi	5
3.1	Why typeclass resolution in Elpi?	5
3.2	From Rocq goals to Elpi proofs	5
3.2.1	Rule encoding	5
3.2.2	Goal encoding	5
3.3	Alternative search indexing	5
3.3.1	Discrimination trees	5
3.3.2	Tabling	5
3.4	Helping higher-order unification	5
3.4.1	η -equivalent terms	5
3.4.2	β -equivalent terms	5
4	Determinacy checking for Elpi	7
4.1	Why determinacy?	7
4.2	Implementation	7
4.2.1	Rule checking	7
4.2.2	Mutual exclusion	7
4.2.3	Caveats	7
4.3	Formalization in Rocq	7

4.3.1	The language	7
4.3.2	The tree semantics	8
4.3.3	The stack semantics	8
4.3.4	Semantics equivalence	8
4.3.5	The main theorem	8
5	Conclusion and Perspectives	9
	Bibliography	11
	List of Figures	13
	Definition's list	15
	Theorem's list	17

Annexes

CHAPTER 1

Introduction

1.1 Prologue

Automation everywhere in CS, we want to set up boilerplates to help our development.

Examples:

- Continuous integration
- Macros are everywhere
- Artificial intelligence

We delegate tasks to some tools.

expert system →

example plus frappant:

artificial intelligence: based on probability but not reproducible, not predicatable

several other examples in computer sciences:

...

1.2 Automation is everywhere

In the domain of formal proof automation is everywhere, and it is formally called elaboration ??

In formal proof, proofs are verbose and have lot of details: types, coercions, canonical structures, type-classes.

In theorem provers the user benefit of notations, implicit inference and tactics to ease his job.

1.2.1 Probabilistic reasoning: artificial intelligence

1.2.2 Automated reasoning: elaboration

Simply typed lambda calculus is an example of automation: an algo checks if ...

1.3 Contributions

In this manuscript we focus on an alternative type-class solver written in the elpi meta-language.

We also talk about a formalized tool. . .

1.3.1 Type-class inference

1.3.2 Static determinacy checking

CHAPTER 2

State of the Art

2.1 The Rocq proof assistant

2.1.1 The basis of the language

2.1.2 Elaboration

In fig. 2.1 (gently offered by Enrico), we have an skemtical representation of the elaboration process.

2.1.3 Ad-hoc polymorphism: Type Classes

One of the most interesting fragment of elaboration is

Cite Haskell, in caml it does not exists: `+. .` vs `+`

2.1.4 Concluding remarks

Overall, Elpi is good for elaboration

2.2 The Elpi language

2.2.1 Controlling backtracking: the `!` operator

2.2.2 Hints about unification

2.2.3 Rocq-Elpi: Rocq and Elpi interleaving

2.2.4 CHR: goal suspension

Tactics, coq-elpi

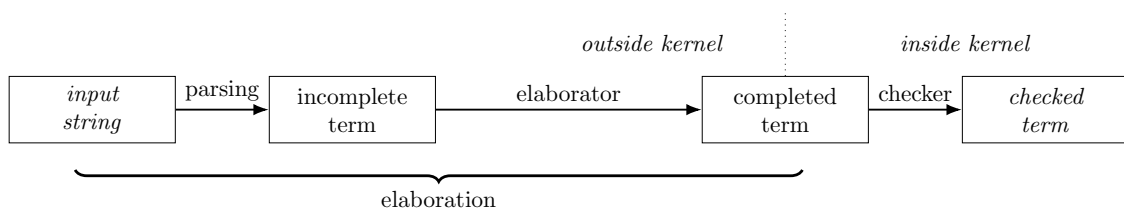


Figure 2.1: Elaboration skeme

Type classes resolution in Elpi

Typeclasses are an elegant way to implement ad-hoc polymorphism [4], so that, the same symbol can be reused in different context while keeping the same mathematical properties. From an implementation point of view, typeclasses are

3.1 Why typeclass resolution in Elpi?

[1]

Rule base automation!

3.2 From Rocq goals to Elpi proofs

3.2.1 Rule encoding

The hooks

3.2.2 Goal encoding

3.3 Alternative search indexing

3.3.1 Discrimination trees

Discrimination trees

TODO: do some benchmarks

Discrimination trees: opt for lists and tree depths.

3.3.2 Tabling

3.4 Helping higher-order unification

3.4.1 η -equivalent terms

3.4.2 β -equivalent terms

[2]

CHAPTER 4

Determinacy checking for Elpi

4.1 Why determinacy?

4.2 Implementation

4.2.1 Rule checking

4.2.2 Mutual exclusion

Using discrimination tree

4.2.3 Caveats

For example, mutual exclusion is not perfect: over approximation of unification

Rule checking ignores flexible rule loading

4.3 Formalization in Rocq

4.3.1 The language

Input must come before outputs

Inputs must come before outputs, with Disc tree this is not mandatory, since unification is weaker.

EG:

```
pred g o:int, o:int.  
  
f 3 3 (g X X).  
f 3 4 (g 3 4).  
  
main:-  
  f X Y (g X Y).
```

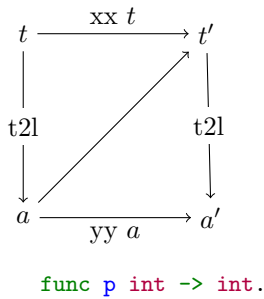
In the code above, both rules can be used with the query, but, the discrimination tree, save the world

4.3.2 The tree semantics**4.3.3 The stack semantics****4.3.4 Semantics equivalence****4.3.5 The main theorem**

CHAPTER 5

Conclusion and Perspectives

[3]



Bibliography

- [1] Davide Fissore and Enrico Tassi. A new Type-Class solver for Coq in Elpi. In *The Coq Workshop*, July 2023. URL: <https://inria.hal.science/hal-04467855>.
- [2] Davide Fissore and Enrico Tassi. Higher-order unification for free!: Reusing the meta-language unification for the object language. In *Proceedings of PPDP*, pages 1–13. ACM, 2024. [doi:10.1145/3678232.3678233](https://doi.org/10.1145/3678232.3678233).
- [3] Davide Fissore and Enrico Tassi. Determinacy checking for elpi: an higher-order logic programming language with cut. In Nada Amin and Joaquín Arias, editors, *Practical Aspects of Declarative Languages*, pages 77–95, Cham, 2026. Springer Nature Switzerland.
- [4] Philip Wadler. Theorems for free! In *Proceedings of FPCA*, FPCA '89, page 347–359. Association for Computing Machinery, 1989. [doi:10.1145/99370.99404](https://doi.org/10.1145/99370.99404).

List of Figures

2.1	Elaboration skeme	3
-----	-----------------------------	---

Definition's list

Theorem's list

Appendix

Tritre francais

Davide FISSORE

Mots-clés : Pâtisserie, Macaron, Ladurée.

Abstract

BLIBLI

Keywords: Pastry, Macaron, Ladurée.